

Simulasi Monitoring Sumber Daya dan Elastisitas Layanan pada Laptop

Gafna Al Faatiha Prabowo (23/513334/PA/21930)

Annisa Salsabila Santiaji (23/514506/PA/21990)

Muhammad Khairul Hasbi (23/519845/PA/22310)

Vincentius Davin Febrillianagata (23/520016/PA/22330)

Department of Computer Science, Universitas Gadjah Mada

Yogyakarta, Indonesia

Email: {gafnaalfaatihaprabowo, annisasalsabilasantiaji, muhammadkhairulhasbi2004, vincentiusdavinfebrillianagata2006}
@mail.ugm.ac.id

TABLE I
KONTRIBUSI ANGGOTA KELOMPOK

Nama	Kontribusi
Gafna Al Faatiha Prabowo	Mengembangkan <code>monitor.py</code> dan <code>load_generator.py</code> serta melakukan pengujian sistem.
Annisa Salsabila Santiaji	Menyusun laporan dan dokumentasi eksperimen.
Muhammad Khairul Hasbi	Menyusun laporan dan dokumentasi eksperimen.
Vincentius Davin Febrillianagata	Mengembangkan <code>service.py</code> serta melakukan pengujian sistem.

I. PENDAHULUAN

Data center modern memerlukan kemampuan untuk memantau kondisi sistem secara berkelanjutan serta menyesuaikan kapasitas layanan sesuai perubahan beban kerja. Dua konsep penting yang mendukung kebutuhan tersebut adalah *resource monitoring* dan *service elasticity*. Monitoring digunakan untuk memperoleh informasi mengenai kondisi sumber daya sistem, sedangkan elastisitas memungkinkan sistem menambah atau mengurangi kapasitas layanan berdasarkan kondisi yang terpantau.

Pada lingkungan data center yang sebenarnya, proses monitoring dan elastisitas umumnya didukung oleh berbagai teknologi seperti virtualisasi, container, orchestration platform, dan sistem monitoring terintegrasi. Namun, konsep dasar kedua mekanisme tersebut dapat dipelajari melalui simulasi sederhana menggunakan perangkat laptop sebagai lingkungan eksperimen.

Project ini bertujuan untuk mensimulasikan monitoring sumber daya dan elastisitas layanan menggunakan beberapa *service instance* yang berjalan pada satu laptop. Monitoring dilakukan dengan mengamati penggunaan CPU dan memori selama sistem menerima beban kerja. Elastisitas disimulasikan melalui proses *scale-out*, yaitu penambahan jumlah service in-

stance saat beban meningkat, serta *scale-in*, yaitu pengurangan jumlah service instance saat beban menurun.

Melalui eksperimen ini, diharapkan dapat diamati hubungan antara jumlah service instance, penggunaan sumber daya, latency, throughput, serta efektivitas mekanisme elastisitas pada lingkungan komputasi berskala kecil.

II. DESAIN SIMULASI

Simulasi dirancang untuk mempelajari konsep monitoring sumber daya dan elastisitas layanan pada lingkungan data center sederhana menggunakan laptop. Sistem yang dibangun terdiri dari tiga komponen utama, yaitu *service instance*, *load generator*, dan *resource monitor*. Ketiga komponen tersebut saling berinteraksi untuk menghasilkan dan mengamati perilaku sistem ketika menerima beban kerja yang berbeda.

Komponen *service instance* diimplementasikan menggunakan `ThreadingHTTPServer` yang berjalan pada port yang berbeda. Setiap instance bertugas menerima request dari pengguna dan melakukan komputasi sederhana berupa perhitungan akar kuadrat secara berulang. Proses komputasi ini digunakan untuk mensimulasikan beban pemrosesan pada server sehingga penggunaan CPU dapat diamati selama eksperimen berlangsung.

Komponen *load generator* digunakan untuk mensimulasikan aktivitas pengguna dengan mengirimkan sejumlah request HTTP ke service instance yang aktif. Distribusi request dilakukan menggunakan mekanisme *round-robin* sehingga beban dapat dibagi secara merata ke seluruh instance yang tersedia. Selain menghasilkan beban kerja, komponen ini juga mengukur metrik performa berupa jumlah request berhasil, jumlah request gagal, rata-rata latency, dan throughput.

Komponen *resource monitor* berfungsi untuk memantau kondisi sumber daya perangkat selama eksperimen berlangsung. Monitoring dilakukan terhadap penggunaan CPU dan memori menggunakan library `psutil`. Data hasil monitoring disimpan dalam format CSV dan divisualisasikan dalam bentuk grafik untuk memudahkan proses analisis.

Eksperimen dilakukan melalui tiga skenario utama, yaitu *baseline*, *scale-out*, dan *scale-in*. Skenario baseline digunakan untuk memperoleh kondisi awal sistem sebelum dilakukan

perubahan kapasitas layanan. Skenario scale-out dilakukan dengan menambah jumlah service instance yang aktif untuk meningkatkan kapasitas layanan ketika beban meningkat. Sebaliknya, skenario scale-in dilakukan dengan mengurangi jumlah instance aktif ketika beban menurun.

Untuk mengamati pengaruh perubahan beban terhadap performa sistem, setiap skenario diuji menggunakan tiga tingkat beban kerja yang berbeda, yaitu beban rendah, sedang, dan tinggi. Beban rendah direpresentasikan dengan 100 request, beban sedang menggunakan 200 request, sedangkan beban tinggi menggunakan 300 request. Pada setiap pengujian dicatat metrik penggunaan CPU, penggunaan memori, latency, throughput, serta tingkat keberhasilan request.

III. TOOLS DAN LINGKUNGAN EKSPERIMEN

Implementasi simulasi dilakukan dengan bahasa pemrograman Python 3 beserta beberapa library pendukung. Library `requests` digunakan untuk mengirim request HTTP pada proses load testing, library `psutil` digunakan untuk memperoleh informasi penggunaan CPU dan memori, sedangkan library `matplotlib` digunakan untuk membuat visualisasi data hasil monitoring. Selain itu, simulasi layanan dikembangkan menggunakan `ThreadingHTTPServer` yang tersedia pada Python Standard Library.

Lingkungan eksperimen terdiri atas dua perangkat laptop yang digunakan untuk menjalankan simulasi dan pengujian. Spesifikasi perangkat yang digunakan ditunjukkan pada tabel

TABLE II
SPESIFIKASI PERANGKAT EKSPERIMEN

Perangkat	Spesifikasi
Laptop 1	Intel Core i5-12450HX (8 Core, 12 Thread), RAM 12 GB DDR5, Windows 11
Laptop 2	Intel Core i5-1235U (10 Core, 12 Thread), RAM 16 GB DDR5, Windows 11

Struktur aplikasi terdiri atas tiga program utama. Program `service.py` digunakan untuk mensimulasikan layanan yang berjalan pada server. Program `load_generator.py` digunakan untuk menghasilkan beban kerja dan mengukur performa layanan, sedangkan program `monitor.py` digunakan untuk melakukan monitoring sumber daya perangkat secara real-time.

Pembagian tugas anggota kelompok pada pengembangan project ini adalah sebagai berikut. Davin bertanggung jawab dalam pengembangan program `service.py`. Gafna bertanggung jawab dalam pengembangan program `load_generator.py` dan `monitor.py`. Salsa dan Hasbi bertanggung jawab dalam penyusunan laporan serta dokumentasi eksperimen.

IV. METODOLOGI EKSPERIMEN

Eksperimen dilakukan untuk mengevaluasi hubungan antara monitoring sumber daya dan elastisitas layanan pada lingkungan komputasi sederhana menggunakan laptop. Sistem terdiri atas service instance, load generator, dan resource monitor yang dijalankan secara lokal.

Service instance menerima request HTTP yang dikirim oleh load generator. Setiap request memicu proses komputasi sederhana untuk menghasilkan beban CPU. Selama eksperimen berlangsung, program monitor mencatat penggunaan CPU dan memori menggunakan library `psutil` setiap satu detik dan menyimpannya ke dalam file CSV untuk dianalisis lebih lanjut.

Eksperimen dilakukan dalam tiga skenario, yaitu baseline, scale-out, dan scale-in. Pada skenario baseline hanya satu service instance yang dijalankan. Pada skenario scale-out jumlah instance ditambah menjadi dua untuk mensimulasikan peningkatan kapasitas layanan. Selanjutnya dilakukan scale-in dengan mengurangi jumlah instance kembali menjadi satu setelah beban menurun.

Setiap skenario diuji menggunakan tiga tingkat beban yang berbeda. Konfigurasi beban yang digunakan ditunjukkan pada Tabel III.

TABLE III
KONFIGURASI BEBAN PENGUJIAN

Tingkat Beban	Total Request
Rendah	100
Sedang	200
Tinggi	300

Parameter yang diamati meliputi rata-rata dan nilai maksimum penggunaan CPU, rata-rata dan nilai maksimum penggunaan memori, latency rata-rata, throughput, jumlah request berhasil, dan jumlah request gagal. Data monitoring yang tersimpan dalam file CSV diolah menggunakan Python untuk menghitung statistik penggunaan resource pada setiap skenario pengujian.

V. HASIL EKSPERIMEN

Eksperimen dilakukan pada dua laptop menggunakan tiga skenario pengujian, yaitu baseline, scale-out, dan scale-in. Setiap skenario diuji menggunakan tiga tingkat beban, yaitu rendah, sedang, dan tinggi. Parameter yang diamati meliputi penggunaan CPU, penggunaan memori, latency rata-rata, throughput, jumlah request berhasil, dan jumlah request gagal.

Tabel IV menunjukkan rata-rata penggunaan CPU dan memori pada kedua laptop selama eksperimen berlangsung.

TABLE IV
RATA-RATA PENGGUNAAN CPU DAN MEMORI

Skenario	Beban	Laptop 1		Laptop 2	
		CPU	Mem	CPU	Mem
Baseline	Rendah	9.86	93.72	2.42	90.43
	Sedang	9.26	92.49	16.41	90.76
	Tinggi	10.69	92.98	16.58	90.25
Scale-Out	Rendah	9.83	93.59	17.33	85.14
	Sedang	8.46	92.97	9.76	83.03
	Tinggi	10.23	94.01	12.26	83.28
Scale-In	Rendah	11.45	94.47	6.92	82.52
	Sedang	9.42	90.12	5.18	81.27
	Tinggi	9.04	89.61	8.75	81.08

Berdasarkan Tabel IV, penggunaan CPU rata-rata pada Laptop 1 berada pada kisaran 8.46–11.45%, sedangkan penggunaan memori berada pada kisaran 89.61–94.47%. Pada Lap-

top 2, penggunaan CPU rata-rata berada pada kisaran 2.42–17.33%, sedangkan penggunaan memori berada pada kisaran 81.08–90.76%.

Perbandingan rata-rata penggunaan CPU pada kedua laptop ditunjukkan pada Gambar 1. Secara umum, penggunaan CPU pada Laptop 1 relatif stabil pada seluruh skenario pengujian. Sementara itu, Laptop 2 menunjukkan variasi penggunaan CPU yang lebih besar, terutama pada skenario baseline dan scale-out.

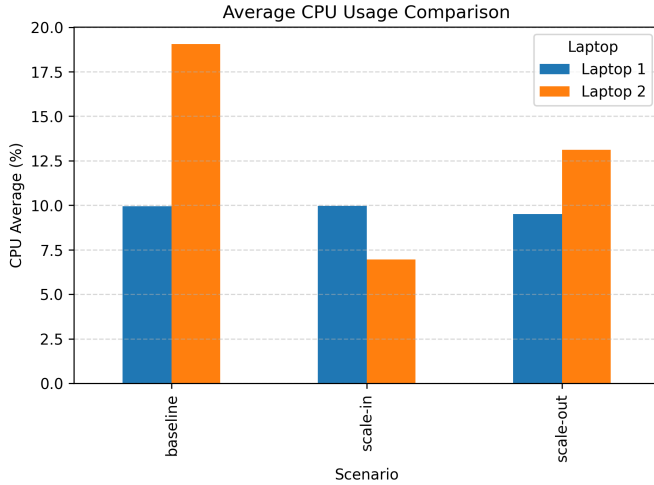


Fig. 1. Perbandingan Rata-rata Penggunaan CPU pada Kedua Laptop

Hasil pengujian performa layanan ditunjukkan pada Tabel V. Seluruh request berhasil diproses tanpa ditemukan request gagal pada seluruh skenario pengujian.

TABLE V
HASIL PENGUJIAN PERFORMA LAYANAN

Beban	Total Request	Success	Latency (s)	Throughput (req/s)
Rendah	100	100	2.06	0.97
Sedang	200	200	2.06	1.94
Tinggi	300	300	2.06	2.43

Latency rata-rata berada pada kisaran 2 detik untuk seluruh tingkat beban. Selain itu, throughput meningkat seiring bertambahnya jumlah request yang diberikan, yaitu dari 0.97 req/s pada beban rendah menjadi 2.43 req/s pada beban tinggi.

Hubungan antara tingkat beban dan throughput ditunjukkan pada Gambar 2. Terlihat bahwa peningkatan jumlah request menghasilkan peningkatan throughput yang relatif konsisten pada seluruh tingkat beban.

Seluruh pengujian berhasil dijalankan dengan tingkat keberhasilan request sebesar 100%. Data hasil eksperimen ini selanjutnya digunakan untuk menganalisis pengaruh perubahan jumlah service instance terhadap penggunaan resource dan performa layanan.

VI. ANALISIS DAN DISKUSI

Berdasarkan hasil eksperimen, penggunaan CPU pada Laptop 1 cenderung stabil pada seluruh skenario pengujian dengan

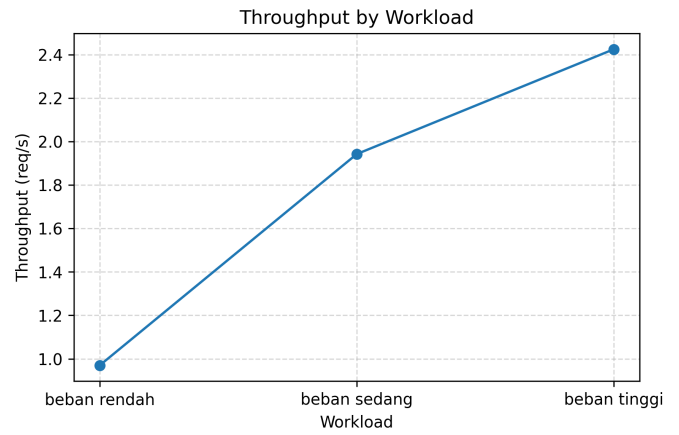


Fig. 2. Throughput pada Berbagai Tingkat Beban

rata-rata berada pada kisaran 8–11%. Sebaliknya, Laptop 2 menunjukkan variasi penggunaan CPU yang lebih besar, terutama pada skenario baseline dan scale-out. Perbedaan ini menunjukkan bahwa karakteristik perangkat keras dan kondisi sistem dapat memengaruhi pola penggunaan resource meskipun konfigurasi eksperimen yang digunakan sama.

Hasil monitoring juga menunjukkan bahwa penggunaan memori cenderung stabil selama eksperimen berlangsung. Tidak terdapat peningkatan penggunaan memori yang signifikan ketika jumlah request maupun jumlah service instance bertambah. Hal ini menunjukkan bahwa beban kerja yang digunakan dalam simulasi lebih banyak memanfaatkan CPU dibandingkan memori.

Pada pengujian performa layanan, seluruh request berhasil diproses dengan tingkat keberhasilan 100%. Selain itu, latency rata-rata berada pada kisaran 2 detik untuk seluruh skenario dan tingkat beban. Hasil ini menunjukkan bahwa layanan mampu mempertahankan waktu respons yang relatif konsisten selama pengujian berlangsung.

Throughput meningkat seiring bertambahnya jumlah request yang diberikan. Pada beban rendah throughput berada pada sekitar 0.97 req/s, kemudian meningkat menjadi 1.94 req/s pada beban sedang dan 2.43 req/s pada beban tinggi. Hasil ini menunjukkan bahwa sistem mampu menangani peningkatan beban kerja tanpa mengalami penurunan performa yang signifikan.

Meskipun jumlah service instance ditambah pada skenario scale-out, peningkatan performa yang diperoleh relatif kecil. Hal ini disebabkan seluruh instance tetap dijalankan pada laptop yang sama sehingga berbagi sumber daya CPU dan memori yang sama. Dengan demikian, penambahan instance tidak menghasilkan penambahan kapasitas komputasi secara fisik, melainkan hanya mendistribusikan beban kerja ke lebih dari satu proses.

Hasil eksperimen ini menunjukkan bahwa monitoring resource memiliki peran penting dalam mengevaluasi kebutuhan elastisitas layanan. Informasi mengenai penggunaan CPU, penggunaan memori, latency, dan throughput dapat

digunakan sebagai dasar untuk menentukan kapan layanan perlu melakukan scale-out atau scale-in. Namun demikian, efektivitas elastisitas sangat dipengaruhi oleh ketersediaan resource fisik yang mendasarinya.

VII. KESIMPULAN

Pada project ini telah berhasil dilakukan simulasi monitoring sumber daya dan elastisitas layanan menggunakan beberapa service instance yang dijalankan pada lingkungan laptop. Monitoring dilakukan terhadap penggunaan CPU dan memori, sedangkan elastisitas layanan disimulasikan melalui skenario baseline, scale-out, dan scale-in dengan berbagai tingkat beban kerja.

Hasil eksperimen menunjukkan bahwa seluruh request dapat diproses dengan tingkat keberhasilan 100%. Penggunaan CPU dan memori berhasil dimonitor selama pengujian berlangsung, sedangkan throughput meningkat seiring bertambahnya jumlah request yang diberikan. Namun, penambahan service instance pada skenario scale-out tidak memberikan peningkatan performa yang signifikan karena seluruh instance tetap berbagi sumber daya fisik yang sama pada satu perangkat.

Berdasarkan hasil tersebut, dapat disimpulkan bahwa monitoring resource dapat digunakan sebagai dasar dalam pengambilan keputusan elastisitas layanan. Meskipun simulasi ini dilakukan pada lingkungan komputasi sederhana, konsep yang diterapkan telah menunjukkan hubungan antara penggunaan resource, performa layanan, dan mekanisme scale-out maupun scale-in yang umum digunakan pada data center modern.

Kode sumber lengkap tersedia di <https://github.com/gafnaa/elasticity-project>.